Math Companion to Soundcalc

October 27, 2026

Work in progress. When in doubt, the code is the source of truth.

Contents

1 Preliminaries **3**
- 1.1 Fields . . . 3
- 1.2 Regime-specific bounds . . . 3
  - 1.2.1 Proximity parameter . . . 3
  - 1.2.2 List sizes . . . 3
  - 1.2.3 Batching errors . . . 3
  - 1.2.4 DEEP-ALI errors . . . 4

2 FRI-based VM security level calculation **5**
- 2.1 FRI parameters . . . 5
- 2.2 Security level for a FRI-based VM . . . 5
- 2.3 Batching Error . . . 6
- 2.4 Commit phase errors . . . 6
- 2.5 Query error . . . 6

3 FRI proof size **7**

4 WHIR-based VM security level calculation **8**
- 4.1 Notation and parameters . . . 8
- 4.2 Bits of Security in UDR and JBR . . . 8
- 4.3 Batching Error ϵ_{batch} . . . 8
- 4.4 Folding Error . . . 9
- 4.5 OOD error . . . 9
- 4.6 Shift Error . . . 9
- 4.7 Final Round Error . . . 10

5 WHIR Proof Size Calculations **11**
- 5.1 Initial Function Size . . . 11
- 5.2 Sumcheck Size for Iteration i . . . 11
- 5.3 Merkle root and OOD Proof Size for Iteration i . . . 11
- 5.4 Query Proof Size for Iteration i . . . 11
- 5.5 Total Proof Size . . . 11

6 STIR-based VM security level calculation **12**
- 6.1 Notation and parameters . . . 12
- 6.2 Bits of Security . . . 12
- 6.3 Batching Error . . . 12
- 6.4 Initial Folding Error . . . 12
- 6.5 OOD Error . . . 12
- 6.6 Shift Error . . . 13
- 6.7 Final Round Error . . . 13

7 STIR Proof Size Calculations **13**

8 Jagged PCS-based VM security level calculation **14**
8.1 Jagged PCS parameters . . . 14
8.2 Security level for a Jagged PCS-based VM . . . 14

9 Jagged PCS proof size **16**
9.1 Sumcheck Proof Sizes . . . 16
9.2 Jagged PCS Proof . . . 16
9.3 Zerocheck Proof . . . 16

10 SWIRL-based VM security level calculation **17**
10.1 SWIRL component bounds . . . 17
10.2 SWIRL WHIR bounds . . . 17
10.3 SWIRL proof size . . . 18

11 Lookup soundness calculation **19**
11.1 Univariate case . . . 19
11.1.1 Multi-column and aggregation: . . . 20
11.1.2 Dummy Variables . . . 20
11.2 Multivariate . . . 20
11.2.1 Multi-column and aggregation: . . . 21
11.3 Grinding . . . 21

1 Preliminaries

1.1 Fields

Fields of size q are denoted as $\mathbb{F}_q$ or simply $\mathbb{F}$. Currently supported prime fields:

- KoalaBear: $q = 2^{31} - 2^{24} + 1$
- Goldilocks: $q = 2^{64} - 2^{32} + 1$
- Mersenne31: $q = 2^{31} - 1$
- BN254: $q = 21888242871839275222246405745257275088548364400416034343698204186575808495617$

1.2 Regime-specific bounds

The proximity parameter δ and list size ℓ depend on the regime (UDR or JBR). These quantities appear in all subsequent error terms.

1.2.1 Proximity parameter

In UDR[1]

$$\delta_U(\rho) = (1 - \rho)/2 \tag{1}$$

In JBR we compute the δ differently.[2]

$$\delta_J(\rho, N, q) = \begin{cases} 1 - \sqrt{\rho} - \frac{\sqrt{\rho}}{100} & \text{if } q > 2^{150} \\ 1 - \sqrt{\rho} - \max(\frac{\rho}{20}, \frac{\sqrt{\rho}}{100}) & \text{otherwise} \end{cases} \tag{2}$$

1.2.2 List sizes

In UDR the list size is trivially[3]

$$\ell = 1$$

In JBR we compute[4]

$$\ell(\rho, N) = \frac{1}{2(1 - \sqrt{\rho} - \delta)\sqrt{\rho}}.$$

1.2.3 Batching errors

We define batching error functions

$$\epsilon_{\text{batch,lin},U}(\rho, N, q), \quad \epsilon_{\text{batch,pow},U}(\rho, N, q, B), \quad \epsilon_{\text{batch,mul},U}(\rho, N, q, B), \quad \epsilon_{\text{batch,lin},J}(\rho, N, q), \quad \epsilon_{\text{batch,pow},J}(\rho, N, q, B)$$

For UDR:

- Compute linear error[5]:
$$\epsilon_{\text{batch,lin},U}(\rho, N, q) = \frac{1 - \rho}{2} \cdot \frac{N}{q\rho} + \frac{1}{q}$$
- Compute powers error:
$$\epsilon_{\text{batch,pow},U}(\rho, N, q, B) = \epsilon_{\text{batch,lin},U}(\rho, N, q) \cdot B$$
- Compute multilinear error:
$$\epsilon_{\text{batch,mul},U}(\rho, N, q, B) = \epsilon_{\text{batch,lin},U}(\rho, N, q) \cdot \lceil \log_2 B \rceil$$

[1] See `proxgaps/unique_decoding.py:get_proximity_parameter()`.
[2] See `proxgaps/johnson_bound.py:get_proximity_parameter()`.
[3] See `proxgaps/unique_decoding.py:get_max_list_size()`.
[4] See `proxgaps/johnson_bound.py:get_max_list_size()`.
[5] See `proxgaps/unique_decoding.py:get_error_linear()`. Code refers to Theorem 4.1 from BCHKS25.

For JBR:

- Compute proximity parameter δ as in (2):
- Compute m[6]:

$$m = 0.5 + \max\left(\left\lceil \frac{\sqrt{\rho}}{1-\sqrt{\rho}-\delta} \right\rceil, 3\right)$$

- Compute linear error[7]:

$$\epsilon_{\text{batch,lin},J}(\rho, N, q) = \frac{N\left(2m^5 + 3m\delta\rho\right) + 3m\rho}{3\rho^{1.5}q}$$

- Compute powers error:

$$\epsilon_{\text{batch,pow},J}(\rho, N, q, B) = \epsilon_{\text{batch,lin},J}(\rho, N, q) \cdot (B-1)$$

1.2.4 DEEP-ALI errors

The DEEP-ALI soundness error[8] (with exception of the list size) and notation are taken from [Hab22b, Section 5.2]:

- Number of AIR constraints [Hab22b, p.15]: $C =$ `num_constraints`
- Overall AIR degree: $d =$ `AIR_max_degree`
- Trace increase constant [Hab22b, Remark 7]: $m_c =$ `max_combo` (set to 2 in [Hab22b])
- Trace length: N (denoted by $|H|$ in [Hab22b])
- List size: ℓ as in section 1.2.2, replaces the L^+ in [Hab22b]

The DEEP-ALI soundness error is computed as follows:

$$\epsilon_{\text{DA}} = \max\left(\frac{\ell \cdot C}{2^{b_{\text{field}}}}, \frac{\ell(d(N+m_c-1)+(N-1))}{2^{b_{\text{field}}} - N - N/\rho}\right) \tag{3}$$

Sanity check (multi-point condition, §4.1.3 in [Hab22b]). Since our DEEP-ALI bound uses multi-point quotients (a.k.a. combo batching), we enforce the domain-sizing requirement

$$N + m_c < (1-\theta) \cdot \frac{N}{\rho}, \tag{4}$$

where ρ is the FRI rate and θ is the proximity parameter determined by the chosen regime. This condition is checked in code via an assertion.[9]

[6] See `proxgaps/johnson_bound.py:get_m()`.
[7] See `proxgaps/johnson_bound.py:get_error_linear()`. Code refers to Theorem 4.2 from BCHKS25.
[8] See `zkvms/circuit.py:_get_DEEP_ALI_errors()`.
[9] See `zkvms/circuit.py:_get_DEEP_ALI_errors()`.

2 FRI-based VM security level calculation

This section calculates the security level for a FRI-based VM in section 2.2.

2.1 FRI parameters

Global parameters used in the FRI analysis:

- r_{FRI} — number of FRI rounds.
- Folding factors $\widehat{\text{folds}} = [k_0, k_1, \ldots, k_{r_{FRI}-1}]$;
- t — number of queries.
- δ — proximity parameter.
- ρ — rate of the Reed-Solomon code.
- N — trace length.
- ℓ — list size.
- $b_{\text{grind},Q}$ — grinding parameter for the query phase.
- n — witness size.
- b_{hash} — number of bits in the hash function output.
- b_{proof} — proof size in bits.
- B — *batch size*. Number of functions appearing in the batched-FRI.
- b_{field} — number of bits in the extension field element.

2.2 Security level for a FRI-based VM

The security level is calculated separately for two regimes: UDR and JBR, using the same procedure.[10]

1. Calculate the FRI round-by-round soundness errors $\mathbf{e}_{\text{FRI}}$:

$$\mathbf{e}_{\text{FRI}} = \max(\mathrm{e}_{\text{batch}}, \{\epsilon_i^{\text{fold}}\}_{i\in[r_{FRI}]}, \mathrm{e}_{\text{query}})$$

2. Obtain optimal δ proximity parameter as by section 1.2.1.
3. Obtain the list size ℓ for the respective δ as by section 1.2.2.
4. Obtain the DEEP-ALI soundness error ϵ_{DA} as by section 1.2.4.
5. Compute the total soundness error as

$$\epsilon = \max(\mathbf{e}_{\text{FRI}}, \epsilon_{\text{DA}}).$$

Then the full security level in bits is the maximum of the two regimes:

$$\text{Security level} = \max(-\log_2 \epsilon_U, -\log_2 \epsilon_J).$$

[10] See `pcs/fri.py:get_pcs_security_levels()`.

2.3 Batching Error

The batching error[11] uses the functions from section 1.2.3.

In the Johnson Bound Regime (JBR) we compute the base error as:

$$\mathrm{e}_{\mathrm{batch}}^{\mathrm{base,JBR}} = \begin{cases} \epsilon_{\mathrm{batch,pow},J}(\rho, N, q, B) & \text{(power batching)}, \\ \epsilon_{\mathrm{batch,lin},J}(\rho, N, q) & \text{(linear batching)}. \end{cases}$$

Whereas in the UDR we compute base error:

$$\mathrm{e}_{\mathrm{batch}}^{\mathrm{base,UDR}} = \begin{cases} \epsilon_{\mathrm{batch,pow},U}(\rho, N, q, B) & \text{(power batching)}, \\ \epsilon_{\mathrm{batch,lin},U}(\rho, N, q) & \text{(linear batching)}, \\ \epsilon_{\mathrm{batch,mul},U}(\rho, N, q, B) & \text{(multilinear batching)}. \end{cases}$$

2.4 Commit phase errors

For round[12] $i \in [r_{FRI}]$ we compute the dimension N_i as

$$N_i = \frac{N}{\prod_{j \in [i]} k_j}$$

Then the folding error in round i is computed as

$$\epsilon_i^{\mathrm{fold}} = \begin{cases} \epsilon_{\mathrm{batch,pow},J}(\rho, N_i, q, k_i), & \text{JBR}, \\ \epsilon_{\mathrm{batch,pow},U}(\rho, N_i, q, k_i), & \text{UDR} \end{cases} \tag{5}$$

2.5 Query error

The query phase error[13] is:

$$\epsilon_{\mathrm{query}} = (1-\theta)^t \cdot 2^{-b_{\mathrm{grind},Q}} \tag{6}$$

[11] See `pcs/pcs.py:_get_batching_error()`.
[12] See `pcs/fri.py:_get_commit_phase_error()`.
[13] See `pcs/fri.py:_get_query_phase_error()`.

3 FRI proof size

The FRI proof[14] contains two parts: Merkle roots, and one "openings" per query, where an "opening" is a Merkle path for each folding layer. For each layer we count the size that this layer contributes, which includes the root and all Merkle paths.

Initial round: one root and one path per query. We assume that for the initial functions, there is only one Merkle root, and each leaf i for that root contains symbols i for all initial functions.

Folding rounds: we assume that "siblings" for the following layers are grouped together in one leaf. This is natural as they always need to be opened together.

The proof size is calculated as follows:

$$b_{\text{proof}} = \underbrace{b_{\text{hash}} + eMP(n, t, B, b_{\text{field}}, b_{\text{hash}})}_{\text{Initial round}} + \underbrace{\sum_{0 \le i < r_{FRI}} \left(b_{\text{hash}} + eMP(\frac{n}{\prod_{0 \le j \le i} \widehat{\text{folds}}[j]}, t, B, b_{\text{field}}, b_{\text{hash}}) \right)}_{\text{Folding rounds}} \quad (7)$$

where $eMP(n, t, b, b_{\text{field}}, b_{\text{hash}})$ is the expected Merkle path size for t queries to the n leaf tree, where each leaf has b elements of b_{field} bits each, calculated as[15]

$$eMP(n, t, b, b_{\text{field}}, b_{\text{hash}}) = \underbrace{t \cdot b \cdot b_{\text{field}}}_{\text{size of all}} + b_{\text{hash}} \sum_{1 \le d \le \lceil \log_2 n \rceil} \lceil 2^d \left((1 - 2^{-d})^t - (1 - 2^{1-d})^t \right) \rceil \quad (8)$$

If the initial functions are committed in several trees of base field columns, with $w_1, \ldots, w_s$ columns each (`initial_tree_widths`), the initial round becomes $\sum_{j=1}^{s} (b_{\text{hash}} + eMP(n, t, w_j, b_{\text{field}}, b_{\text{hash}}))$.

The n_{evals} out-of-domain evaluations sent before FRI (`num_evals`) add $n_{\text{evals}} \cdot b_{\text{field}}$ bits to the proof size, also in STIR.[16]

[14] See `pcs/fri.py:get_FRI_proof_size_bits()`.

[15] See `common/utils.py:get_size_of_merkle_multi_proof_bits_expected()`.

[16] See `pcs/fri.py:_get_proof_size_bits()`.

4 WHIR-based VM security level calculation

4.1 Notation and parameters

- Number of iterations: $M = \texttt{num_iterations}$
- Iteration index: $i \in \{0, \ldots, M-1\}$
- Folding factor in iteration i: $k_i = \texttt{folding_factors}[i]$
- Folding round index in iteration i: $s \in \{0, \ldots, k_i - 1\}$
- Extension field size: $|\mathbb{F}| = b_{\text{field}}$
- Constraint degree: $d = \texttt{constraint_degree}$
- Log-degree in iteration i: $m_i = \texttt{log_degrees}[i]$, and we set $m_{i+1} = m_i - k_i$ (equivalently $m_i = m_0 - \sum_{j=0}^{i-1} k_j$)
- Log-inverted rate in iteration i: $\mu_i = \texttt{log_inv_rates}[i]$, and we set $\mu_{i+1} = \mu_i + (k_i - 1)$ (equivalently $\mu_i = \log_2 \frac{1}{\rho} + \sum_{j=0}^{i-1}(k_j - 1)$)
- Iteration-round-specific RS codes:

$$\mathcal{C}_{i,s} = (\mathcal{C}_{i,s}[\rho], \mathcal{C}_{i,s}[N])_{i,s} = (2^{-\mu_i}, 2^{m_i - s})$$

- OOD sample counts (length $M-1$): $w_j = \texttt{num_ood_samples}[j]$ for $j \in \{0, \ldots, M-2\}$; iteration $i \in \{1, \ldots, M-1\}$ sends w_{i-1} OOD evaluations.
- Number of queries in iteration i: $t_i = \texttt{num_queries}[i]$ for $i \in [M]$.
- Grinding bits:

$$g_{\text{batch}}, \quad \{g_{i,s}^{\text{fold}}\}_{i \in [M],\, s \in [k_i]}, \quad \{g_j^{\text{ood}}\}_{j \in \{0,\ldots,M-2\}}, \quad \{g_i^{\text{qry}}\}_{i \in [M]}.$$

4.2 Bits of Security in UDR and JBR

In order to compute the bits of security in UDR and JBR regimes, we compute the following error terms using the parameters from section 4.1.

For any[17] error term:

$$\lambda(\epsilon) = \lfloor -\log_2 \epsilon \rfloor.$$

Overall security:

$$\lambda_{\text{total}} = \min\{\lambda_{\text{batch}}, \{\lambda_{i,s}^{\text{fold}}\}_{i \in [M],\, s \in [k_i]}, \{\lambda_i^{\text{out}}\}_{i \in \{1,\ldots,M-1\}}, \{\lambda_i^{\text{shift}}\}_{i \in \{1,\ldots,M-1\}}, \lambda^{\text{fin}}\}.$$

4.3 Batching Error ϵ_{batch}

The batching error[18] uses parameters from section 4.1.

In the Johnson Bound Regime (JBR) we proceed as follows:

- Set code parameters

$$(\rho, N) = \left(2^{-\mu_0}, 2^{m_0}\right)$$

- Compute base error (cf. section 1.2.3):

$$\varepsilon_{\text{batch},JBR}^{\text{base}} = \begin{cases} \epsilon_{\text{batch,pow},J}(\rho, N, b_{\text{field}}, B) & \text{(power batching)}, \\ \epsilon_{\text{batch,lin},J}(\rho, N, b_{\text{field}}) & \text{(linear batching)}. \end{cases}$$

Whereas in the UDR we do as follows:

[17] See `pcs/whir.py:get_pcs_security_levels()`.

[18] See `pcs/pcs.py:_get_batching_error()`.

- Compute base error (cf. section 1.2.3):

$$\varepsilon^{\text{base}}_{\text{batch},UDR} = \begin{cases} \epsilon_{\text{batch,pow},U} & \text{(power batching)}, \\ \epsilon_{\text{batch,lin},U} & \text{(linear batching)}. \end{cases}$$

In both regimes we do as follows after grinding:

$$\epsilon_{\text{batch}} = \varepsilon^{\text{base}}_{\text{batch}} \cdot 2^{-g_{\text{batch}}}.$$

4.4 Folding Error

The folding error per round refers to Theorem 5.2 of the WHIR paper.[19]

For iteration $i \in [M]$ and folding round $s \in [k_i]$ in JBR we do[20]:

- Get list size as in section 1.2.2

$$\ell_{i,s} = \ell(\mathcal{C}_{i,s}[\rho])$$

- Base error (two terms):

$$\varepsilon^{\text{fold,base,J}}_{i,s} = d \cdot \frac{\ell_{i,s}}{b_{\text{field}}} + \epsilon_{\text{batch,pow},J}(\mathcal{C}_{i,s+1}[\rho], \mathcal{C}_{i,s+1}[N], b_{\text{field}}, B = 2).$$

And in UDR the base error is

$$\varepsilon^{\text{fold,base,U}}_{i,s} = \frac{d}{b_{\text{field}}} + \epsilon_{\text{batch,pow},U}(\mathcal{C}_{i,s+1}[\rho], \mathcal{C}_{i,s+1}[N], b_{\text{field}}, B = 2).$$

After grinding:

$$\epsilon^{\text{fold}}_{i,s} = \varepsilon^{\text{fold,base}}_{i,s} \cdot 2^{-g^{\text{fold}}_{i,s}}.$$

4.5 OOD error

The Out-of-Domain error[21] is computed as follows.

For iteration $i \in \{1, 2, \ldots, M-1\}$

- Base error in JBR (cf. section 1.2.2):

$$\varepsilon^{\text{out,base,J}}_{i} = \ell^2_{i,0} \left(\frac{2^{m_i}}{2q}\right)^{w_{i-1}}.$$

- Base error in UDR:

$$\varepsilon^{\text{out,base,U}}_{i} = \left(\frac{2^{m_i}}{2q}\right)^{w_{i-1}}.$$

- After grinding:

$$\epsilon^{\text{out}}_{i} = \varepsilon^{\text{out,base}}_{i} \cdot 2^{-g^{\text{ood}}_{i-1}}.$$

4.6 Shift Error

For iteration[22] $i \in \{1, 2, \ldots, M-1\}$ in JBR

- Get δ for the iteration using (2):

$$\delta_i = \min_{s \in [k_{i-1}+1]} \delta_J(\mathcal{C}_{i-1,s}[\rho], \mathcal{C}_{i-1,s}[N], q)$$

[19] See `pcs/whir.py:_epsilon_fold()`.

[20] The function `pcs/whir.py:_epsilon_fold()` counts folding rounds from 1 to k_i, whereas here we count from 0 to $k_i - 1$.

[21] See `pcs/whir.py:_epsilon_out()`.

[22] See `pcs/whir.py:_epsilon_shift()`.

- Base error (two terms):

$$\varepsilon_i^{\text{shift,base}} = (1-\delta_i)^{t_{i-1}} + \ell_{i,0} \cdot \frac{t_{i-1}+1}{q}.$$

Same procedure in UDR:

- Get δ for the iteration using (1):

$$\delta_i = \min_{s \in [k_{i-1}+1]} \delta_U(\mathcal{C}_{i-1,s}[\rho])$$

- Base error :

$$\varepsilon_i^{\text{shift,base}} = (1-\delta_i)^{t_{i-1}} + \frac{t_{i-1}+1}{q}.$$

After grinding:

$$\epsilon_i^{\text{shift}} = \varepsilon_i^{\text{shift,base}} \cdot 2^{-g_{i-1}^{\text{qry}}}.$$

4.7 Final Round Error

In JBR:[23]

- Get δ for the iteration using (2):

$$\delta_{M-1} = \min_{s \in [k_{M-1}+1]} \delta_J(\mathcal{C}_{M-1,s}[\rho])$$

UDR:

- Get δ for the iteration using (1):

$$\delta_{M-1} = \min_{s \in [k_{M-1}+1]} \delta_U(\mathcal{C}_{M-1,s}[\rho])$$

Then the base error is :

$$\varepsilon^{\text{fin,base}} = (1-\delta_{M-1})^{t_{M-1}}.$$

After grinding:

$$\epsilon^{\text{fin}} = (1-\delta_{M-1})^{t_{M-1}} \cdot 2^{-g_{M-1}^{\text{qry}}}.$$

[23] See `pcs/whir.py:_epsilon_final()`.

5 WHIR Proof Size Calculations

This section summarizes the proof-size formula.[24]

5.1 Initial Function Size

$$S_{\text{if}} = b_{\text{hash}}.$$

5.2 Sumcheck Size for Iteration i

The folding proof size at round i is:

$$S_{\text{sumcheck},i} = k_i(d-1)b_{\text{field}}.$$

5.3 Merkle root and OOD Proof Size for Iteration i

For iterations $i \in \{1, \ldots, M-1\}$, the prover sends a Merkle root for f_i and the w_{i-1} OOD evaluations:

$$S_{\text{ood},i} = b_{\text{hash}} + w_{i-1}b_{\text{field}}.$$

After the last iteration the prover also sends the final multilinear polynomial:

$$S_{\text{final}} = 2^{m_M} b_{\text{field}}.$$

5.4 Query Proof Size for Iteration i

We compute

$$S_{\text{qry},i} = \begin{cases} eMP(2^{m_i+\mu_i-k_i}, t_i, 2^{k_i}, b_{\text{field}}, b_{\text{hash}}). & i > 0 \\ eMP(2^{m_0+\mu_0-k_0}, t_0, B \cdot 2^{k_0}, b_{\text{field}}, b_{\text{hash}}). & i = 0 \end{cases}$$

5.5 Total Proof Size

Collecting all terms and summing over all M iterations:

$$\boxed{S_{\text{total}} = S_{\text{if}} + S_{\text{final}} + \sum_{i \in [M]} (S_{\text{sumcheck},i} + S_{\text{qry},i}) + \sum_{i \in \{1,\ldots,M-1\}} S_{\text{ood},i}}$$

[24] See `pcs/whir.py:_get_proof_size_bits()`.

6 STIR-based VM security level calculation

This section describes the computation in `soundcalc/pcs/stir.py`, which follows the round-by-round soundness analysis of STIR [ACFY24] (Lemma 5.4 / Section 5.2). STIR shares its degree-shift and rate schedule with WHIR (section 4), but has no sumcheck: each iteration folds by 2^k with a single challenge, and the degree correction is done by combining quotients with powers of a challenge.

6.1 Notation and parameters

- Number of iterations: $M = \texttt{num_iterations}$, iteration index $i \in \{0, \dots, M-1\}$
- Extension field size: $|\mathbb{F}| = b_{\text{field}}$
- Log folding factors: $k_i = \texttt{folding_factors}[i]$, $i \in \{0, \dots, M-1\}$; iteration i folds 2^{k_i} evaluations at once
- Log-degree in iteration i: $m_0 = \texttt{log_degree}$, $m_{i+1} = m_i - k_i$, for $i \in \{0, \dots, M-1\}$
- Log-inverted rate in iteration i: $\mu_0 = \log_2 \frac{1}{\rho}$, $\mu_{i+1} = \mu_i + k_i - 1$
- Codes of iteration i: $\mathcal{C}_i = (2^{-\mu_i}, 2^{m_i})$ and the folded code $\mathcal{C}_i^{\text{fold}} = (2^{-\mu_i}, 2^{m_i - k_i})$ (same rate, since degree and domain both shrink by 2^{k_i})
- Number of OOD samples for f_i, $i \in \{1, \dots, M-1\}$: $s_i = \texttt{num_ood_samples}[i-1]$
- Number of queries into f_i: $t_i = \texttt{num_queries}[i]$
- Grinding bits: g_{batch}, g^{fold}, $\{g_i^{\text{ood}}\}$, $\{g_i^{\text{qry}}\}$
- Proximity parameter and list size of iteration i (from the regime, cf. section 1.2.2):

$$\delta_i = \min\{\delta(\mathcal{C}_i), \delta(\mathcal{C}_i^{\text{fold}})\}, \qquad \ell_i = \ell(\mathcal{C}_i[\rho]).$$

- $\text{err}^*(\mathcal{C}, B)$ denotes the regime's proximity-gaps error for combining B functions with powers of a single challenge (`get_error_powers`), i.e. $\epsilon_{\text{batch,pow},J}(\mathcal{C}[\rho], \mathcal{C}[N], b_{\text{field}}, B)$ in JBR and $\epsilon_{\text{batch,pow},U}$ in UDR.

6.2 Bits of Security

Overall security is

$$\lambda_{\text{total}} = \min\{\lambda_{\text{batch}}, \lambda_0^{\text{fold}}, \{\lambda_i^{\text{out}}\}_{i\in\{1,\dots,M-1\}}, \{\lambda_i^{\text{shift}}\}_{i\in\{1,\dots,M-1\}}, \lambda^{\text{fin}}\}, \qquad \lambda(\epsilon) = \lfloor -\log_2 \epsilon \rfloor.$$

6.3 Batching Error

`pcs/pcs.py:_get_batching_error()`. Identical to WHIR on the code $\mathcal{C}_0$:

$$\epsilon_{\text{batch}} = \varepsilon_{\text{batch}}^{\text{base}}(\mathcal{C}_0) \cdot 2^{-g_{\text{batch}}},$$

with $\varepsilon_{\text{batch}}^{\text{base}}$ as in section 1.2.3 (powers or linear batching).

6.4 Initial Folding Error

`_epsilon_fold()`. Folding f_0 by 2^{k_0} with the powers $1, r, \dots, r^{2^{k_0}-1}$ of a single challenge:

$$\epsilon_0^{\text{fold}} = \text{err}^*(\mathcal{C}_0^{\text{fold}}, 2^{k_0}) \cdot 2^{-g^{\text{fold}}}.$$

6.5 OOD Error

`_epsilon_out()`. For $i \in \{1, \dots, M-1\}$:

$$\epsilon_i^{\text{out}} = \frac{\ell_i^2}{2} \left(\frac{2^{m_i}}{b_{\text{field}} - |L_i|} \right)^{s_i} \cdot 2^{-g_{i-1}^{\text{ood}}}, \qquad |L_i| = 2^{m_i + \mu_i}.$$

The OOD point is sampled from $\mathbb{F} \setminus L_i$, hence the denominator. For $s_i = 1$ this is the term of Lemma 5.4; for $s_i > 1$ we follow `stir-whir-scripts` and raise only the collision probability to the power s_i. In UDR, $\ell_i = 1$.

6.6 Shift Error

`_epsilon_shift()`. For $i \in \{1, \ldots, M-1\}$ the shift error has three terms: the t_{i-1} queries into f_{i-1} miss the disagreement, the degree correction (combining the $t_{i-1} + s_i$ quotient answers by powers), and the folding of f_i:

$$\epsilon_i^{\text{shift}} = \left((1-\delta_{i-1})^{t_{i-1}} + \text{err}^*(\mathcal{C}_i, t_{i-1}+s_i) + \text{err}^*(\mathcal{C}_i^{\text{fold}}, 2^{k_i})\right) \cdot 2^{-g_{i-1}^{\text{qry}}}.$$

6.7 Final Round Error

`_epsilon_final()`:

$$\epsilon^{\text{fin}} = (1-\delta_{M-1})^{t_{M-1}} \cdot 2^{-g_{M-1}^{\text{qry}}}.$$

7 STIR Proof Size Calculations

Computed in `get_STIR_proof_size_bits()`. As in FRI, the batched f_0 is committed in its own tree. Prover messages are: the Merkle roots of the B batched functions and of f_0; for $i \in \{1, \ldots, M-1\}$ the Merkle root of f_i and the s_i OOD answers; the final polynomial of degree $< 2^{m_M}$ in the clear; a Merkle multi-proof opening the B batched (base field) functions at t_0 points; and for every f_i, $i \in \{0, \ldots, M-1\}$, a Merkle multi-proof opening t_i leaves, where a leaf holds a folding block of 2^{k_i} elements:

$$S_{\text{total}} = 2b_{\text{hash}} + \sum_{i=1}^{M-1} (b_{\text{hash}} + s_i b_{\text{field}}) + 2^{m_M} b_{\text{field}} + eMP(2^{m_0+\mu_0}, t_0, B, b_{\text{field}}, b_{\text{hash}}) + \sum_{i=0}^{M-1} eMP(2^{m_i+\mu_i-k_i}, t_i, 2^{k_i}, b_{\text{field}}, b_{\text{hash}})$$

With `initial_tree_widths` $(w_1, \ldots, w_s)$, the batched functions live in s trees instead: their root and $eMP(2^{m_0+\mu_0}, t_0, B, b_{\text{field}}, b_{\text{h}}$ become $\sum_{j=1}^{s} (b_{\text{hash}} + eMP(2^{m_0+\mu_0}, t_0, w_j, b_{\text{field}}, b_{\text{hash}}))$.

8 Jagged PCS-based VM security level calculation

This section calculates the security level for a Jagged PCS-based VM in section 8.2.

8.1 Jagged PCS parameters

Global parameters used in the analysis:

- r_{FRI} — number of FRI rounds.
- Folding factors $\widehat{\text{folds}} = [k_0, k_1, \ldots, k_{r_{FRI}-1}]$;
- t — number of queries.
- δ — proximity parameter.
- ρ — rate of the Reed-Solomon code.
- $b_{\text{grind},Q}$ — grinding parameter for the query phase.
- N_{trace} — maximum height of the trace.
- B_{trace} — number of columns of the trace.
- N_{stack} — height of the dense representation of the trace
- B_{stack} — number of columns in the dense representation of the trace
- d — constraint degree
- C — number of constraints
- b_{hash} — number of bits in the hash function output.
- b_{proof} — proof size in bits.
- b_{field} — number of bits in the extension field element.

8.2 Security level for a Jagged PCS-based VM

The security level is calculated only for the unique decoding regime.

The soundness error from FRI outside of the DEEP-ALI error can be analyzed exactly as batched FRI, with trace length N_{stack} and batch size B_{stack}. One of the other soundness error comes from zerocheck, which checks

$$\sum_{x \in H} \left(\text{eq}(r, x) \cdot \sum_{i=1}^{C} \lambda^i \cdot C_i(t_1(x), t_2(x), \cdots, t_B(x)) \right) = 0$$

where $H = \{0,1\}^{\log N_{\text{trace}}}$ is the boolean hypercube.

Following the analysis in Appendix A.2 of [Gru24], the random linear combination from $\text{eq}(r, x)$ adds a $(\log N_{\text{trace}})/q$ soundness error. The random linear combination from λ^i adds a C/q soundness error. Finally, the sumcheck routine adds a $((d+1)\log N_{\text{trace}})/q$ soundness error. Combined, the overall soundness error from the zerocheck is at most

$$\frac{C + (d+2)\log N_{\text{trace}}}{q}$$

The remaining soundness error comes from reducing the evaluation claims for the zerocheck into a single evaluation claim for the dense representation.[25]

First, the random linear combination using the eq polynomial for the B_{trace} evaluations add an at most

[25] See `pcs/jagged.py:_get_reduction_error()`.

$$\frac{\log B_{\text{trace}}}{q}$$

soundness error, and the jagged evaluation and jagged sumcheck combined adds an at most

$$\frac{6 \cdot (\log N_{\text{stack}} + \log B_{\text{stack}}) + 4}{q}$$

soundness error, as it is a degree 2 sumcheck with $2 \cdot \log \text{trace} + 2$ variables and a degree 2 sumcheck with $\log \text{trace}$ variables. Here, $\log \text{trace} = \log(N_{\text{stack}} \cdot B_{\text{stack}}) = \log N_{\text{stack}} + \log B_{\text{stack}}$.

These bounds are pessimistic, and the round-by-round soundness errors are lower.

9 Jagged PCS proof size

As in the security level calculation, the underlying dense PCS proof size can be analyzed with FRI's proof size with trace length N_{stack} and batch size B_{stack}. Here, sizes of the other parts of the PCS proof are discussed.

9.1 Sumcheck Proof Sizes

Given a sumcheck of degree d and number of variables n, the size of the sumcheck proof is as follows.

- univariate polynomials: n degree d polynomials, so $n(d+1)b_{\text{field}}$.
- claimed sum: b_{field}.
- evaluation point and result: $nb_{\text{field}} + b_{\text{field}} = (n+1)b_{\text{field}}$.

So the overall size is

$$\text{Sumcheck}(d, n) = (n(d+2)+2)b_{\text{field}}$$

9.2 Jagged PCS Proof

The jagged sumcheck and jagged evaluation proofs lead to a sumcheck proofs of size

$$\text{Sumcheck}(2, \log \text{trace}) + \text{Sumcheck}(2, 2 \cdot \log \text{trace} + 2)$$

where $\text{trace} = N_{\text{stack}} \cdot B_{\text{stack}}$ is the size of the trace.

9.3 Zerocheck Proof

The zerocheck proof with AIR degree d, is a sumcheck of degree $d+1$ with $\log N_{\text{trace}}$ variables, so it has size

$$\text{Sumcheck}(d+1, \log N_{\text{trace}})$$

10 SWIRL-based VM security level calculation

The SWIRL proof system is used by OpenVM2. It is a matrix-stacking proof system that reduces trace evaluations to a stacked polynomial evaluation, which is then opened via the WHIR PCS (section 4).

This companion summarizes the bounds implemented by soundcalc. For full details, see:

- SWIRL paper: `https://openvm.dev/swirl.pdf`
- Canonical OpenVM2 soundness calculator: `https://github.com/openvm-org/stark-backend/blob/develop-v2/crates/stark-backend/src/soundness.rs`

The implementation follows the canonical calculator round by round.[26] Let L_{PCS} be the list-size bound for the initial WHIR proximity regime. For unique decoding, $\log_2 L_{\mathrm{PCS}} = 0$. For list decoding, the BCHKS25 bound used for WHIR determines $\log_2 L_{\mathrm{PCS}}$.

10.1 SWIRL component bounds

LogUp soundness includes the PCS list-size union bound and effective proof-of-work bits g_{eff}, where g_{eff} accounts for the small `sample_bits` bias over the BabyBear base field:[27]

$$\lambda_{\mathrm{LogUp}} = \log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2(2I_{\max}) - \log_2 M_{\max} - \log_2 L_{\mathrm{PCS}} + g_{\mathrm{eff}}.$$

The ordinary GKR checks use the same worst per-round bounds as the backend:

$$\lambda_{\mathrm{GKR,sumcheck}} = \log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2 3, \qquad \lambda_{\mathrm{GKR,batching}} = \log_2 |\mathbb{F}_{\mathrm{ext}}|.$$

ZeroCheck after the fused input-layer boundary has degree

$$d_{\mathrm{zc}} = \max\{(d_{\max}+1)(2^{\ell_{\mathrm{skip}}}-1),\, d_{\max}+1\},$$

so[28]

$$\lambda_{\mathrm{ZeroCheck}} = \log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2 d_{\mathrm{zc}} - \log_2 L_{\mathrm{PCS}}.$$

The fused boundary and batch-sumcheck batching term uses[29]

$$d_{\mathrm{fused}} = \max\{3, n_{\mathrm{extra}}\} + (2^{\ell_{\mathrm{skip}}}-1) + (C_{\max}-1), \qquad d_{\mathrm{batch}} = 3A - 1,$$

where $C_{\max}$ is the maximum number of constraints in one AIR and A is the number of AIRs. The security is

$$\lambda_{\mathrm{fused}} = \min\{\log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2 d_{\mathrm{fused}}, \log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2 d_{\mathrm{batch}}\} - \log_2 L_{\mathrm{PCS}}.$$

Stacked reduction uses the minimum of trace-column batching, the univariate round, and multilinear rounds, again with the PCS list-size union bound:

$$\lambda_{\mathrm{stack}} = \min\{\log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2(2N_{\mathrm{cols}}), \log_2 |\mathbb{F}_{\mathrm{ext}}| - \log_2(2(2^{\ell_{\mathrm{skip}}}-1)), \log_2 |\mathbb{F}_{\mathrm{ext}}| - 1\} - \log_2 L_{\mathrm{PCS}}.$$

10.2 SWIRL WHIR bounds

SWIRL uses the OpenVM backend WHIR soundness calculation rather than the older generic WHIR routine.[30] Each folding sub-round combines the WHIR sumcheck error with the proximity-gap error by summing the two errors before converting to bits:

$$\lambda_{\mathrm{fold}} = -\log_2(2^{-\lambda_{\mathrm{sumcheck}}} + 2^{-\lambda_{\mathrm{gap}}}).$$

Each shift or final round similarly combines query sampling with in-domain γ-batching:

$$\lambda_{\mathrm{shift}} = -\log_2(2^{-\lambda_{\mathrm{query}}} + 2^{-\lambda_{\gamma}}).$$

The WHIR contribution is the minimum of μ-batching, all folding round-by-round bounds, all shift/final bounds, and non-final OOD bounds. With a single stacked column there is no μ-batching round, so the μ-batching term vanishes.

[26] See `soundcalc/circuits/swirl/calculator.py:calculate_swirl_soundness()`.

[27] See `soundcalc/circuits/swirl/calculator.py:SWIRLLogUpSecurityParameters.get_soundness_bits()` and `soundcalc/circuits/swirl/calculator.py:_effective_pow_bits()`.

[28] See `soundcalc/circuits/swirl/calculator.py:calculate_zerocheck_sumcheck_soundness()`.

[29] See `soundcalc/circuits/swirl/calculator.py:calculate_constraint_batching_soundness()`.

[30] See `soundcalc/circuits/swirl/calculator.py:_calculate_whir_soundness()`.

10.3 SWIRL proof size

For OpenVM2, soundcalc estimates SWIRL proof size by counting the backend proof codec fields, not by applying the generic WHIR proof-size model alone.[31] The total proof size is decomposed as

$$b_{\text{proof}} = b_{\text{pre}} + b_{\text{GKR}} + b_{\text{batch}} + b_{\text{stack}} + b_{\text{WHIR}}.$$

The terms correspond to the top-level fields of `stark-backend/src/proof.rs`: the proof preamble, LogUp GKR proof, batch-constraint proof, stacking proof, and backend WHIR proof. Base-field elements are counted by their encoded byte length; for BabyBear this is 32 bits even though the field modulus has 31 bits.

The OpenVM2 configuration uses proof-size bounds that assume all optional AIRs are present. If no explicit `proof_size_*` value is provided for a circuit, the proof-size calculator uses the matching `soundness_*` bound where available.

[31] See `soundcalc/circuits/swirl/proof_size.py:get_swirl_proof_size_breakdown_bits()`.

11 Lookup soundness calculation

We consider logUP, the lookup argument introduced in [Hab22a].

We have a table $\mathbf{T} \in \mathbb{F}^{T \times S}$ and tables $\mathbf{L}^1, \ldots, \mathbf{L}^K$ in $\mathbb{F}^{L \times S}$ that are lookups of it, that is, $\forall\ i \in [L]\ \exists\ j \in [T]\ s.t.\ \mathbf{L}_i = \mathbf{T}_j$.

For the sake of the integration and in order to be consistent with the implementation parameters, we consider separately the cases when the lookup argument is proven using a univariate or multivariate IOP. We also consider a different case when $S = 1$.

In the code, the parameter controlling the aggregation is `multilinear_fingerprint`. If it is not specified, then we assume it follows the lookup type:

- univariate lookup $\Rightarrow$ univariate aggregation,
- multivariate lookup $\Rightarrow$ multivariate aggregation.

Only when `multilinear_fingerprint` is explicitly specified do we override this default.

The logUP argument proves lookup relations by proving that given a vector $\vec{m} \in \mathbb{F}^{|\mathbf{T}|}$ such that $m_j \in \mathbb{Z}$, the lookup relation holds if and only if the following equation is satisfied.

$$\sum_{\vec{l} \in \mathbf{L}} \frac{1}{X - \vec{l}} = \sum_{\vec{t} \in \mathbf{T}} \frac{m_{\vec{t}}}{X - \vec{t}}$$

The expression above is equivalent to considering only one table $\mathbf{L}$ and having each element taken a positive or negative value depending on whether it is sent from the current trace or received from some previous computation. In that case, we just set $\mathbf{L} = \mathbf{0}$ and have

$$\sum_{\vec{t} \in \mathbf{T}} \frac{m_{\vec{t}}}{X - \vec{t}} = 0$$

We split the logUP protocol in two steps:

1. Compute some polynomial expression of the equation above
2. Prove consistency with $\mathbf{T}$ and $\mathbf{L}$

11.1 Univariate case

In this case, we always assume that proving the well formation of the sums is deferred to AIR constraints, and that the additional constraints this causes are already taken into account in the size of the AIR trace for the soundness of the proof of correct execution of it.

If `multilinear_fingerprint` is not specified, then in this section we assume it is `false`, i.e. the aggregation is univariate.

We consider the following parameters:

- F = size of the extension field $\mathbb{F}$
- H_L = size of the set used to interpolate $\mathbf{L}$
- H_T = size of the set used to interpolate $\mathbf{T}$
- $H = H_L + H_T$
- S = number of columns of $\mathbf{T}$ and $\mathbf{L}$
- K = number of lookups performed on $\mathbf{T}$

In the soundcalc code and TOML configs, H_L and H_T correspond to the fields `rows_L` and `rows_T` respectively.

The expression

$$\sum_{i=1}^{L} \frac{1}{X - \sum_{s=1}^{S} \mathbf{L}_{is} Y^{s-1}} = \sum_{j=1}^{T} \frac{m_j}{X - \sum_{s=1}^{S} \mathbf{T}_{js} Y^{s-1}}$$

is replaced, upon receiving α, β uniformly sampled from $\mathbb{F}$, by the polynomial equality

$$\sum_{i=1}^{L} \frac{1}{\alpha - \sum_{s=1}^{S} \mathbf{L}_{is}\beta^{s-1}} \lambda_i(X) = \sum_{j=1}^{T} \frac{m_j}{\alpha - \sum_{s=1}^{S} \mathbf{T}_{js}\beta^{s-1}} \mu_j(X)$$

$\lambda_i(X), \mu_j(X)$ being the Lagrange interpolation polynomials of some domains of size L and T, respectively.

The probability that a prover gets a randomness that verifies without $\mathbf{L}$ being a lookup on $\mathbf{T}$ is bounded by ϵ_{sum} as follows:

11.1.1 Multi-column and aggregation:

Let R be the contribution of the column aggregation. In the default univariate setting of this section, we take $R = S$. If the implementation explicitly overrides the aggregation to be multivariate, then we instead take $R = \log(S)$. Thus:

$$R = \begin{cases} S & \text{if the aggregation is univariate,} \\ \log(S) & \text{if the aggregation is multivariate.} \end{cases}$$

Then the lookup soundness error is bounded by

$$\epsilon_{\mathsf{sum}} \leq \frac{KHR}{F}.$$

For the single-column case, we take $R = 1$.

11.1.2 Dummy Variables

If $\mathbf{T}, \mathbf{L}$ are smaller than the interpolation set and the tables are filled with dummy variables, those variables should be taken into account in the sizes of H_L and H_T.

11.2 Multivariate

If `multilinear_fingerprint` is not specified, then in this section we assume it is `true`, i.e. the aggregation is multivariate.

We consider the following parameters:

- F = size of the extension field $\mathbb{F}$
- H_L = size of the alphabet used to interpolate $\mathbf{L}$
- H_T = size of the alphabet used to interpolate $\mathbf{T}$
- $H = H_L + H_T$
- S = number of columns of $\mathbf{T}$ and $\mathbf{L}$
- K = number of lookups performed on $\mathbf{T}$

The expression

$$\sum_{i=1}^{L} \frac{1}{X - \sum_{s=1}^{S} \mathbf{L}_{is}Y^{s-1}} = \sum_{j=1}^{T} \frac{m_j}{X - \sum_{s=1}^{S} \mathbf{T}_{js}Y^{s-1}}$$

is replaced, upon receiving α uniformly sampled from $\mathbb{F}$ and $\vec{\beta}$ uniformly sampled from $\mathbb{F}^{\log S}$, by the polynomial equation:

$$\sum_{\vec{x} \in H} \frac{1}{\alpha - L(\vec{x}, \vec{\beta})} = \sum_{\vec{x} \in H} \frac{m(\vec{x})}{\alpha - T(\vec{x}, \vec{\beta})}$$

To convert the equation above into a polynomial one, both sides are multiplied by

$$\prod_{\vec{x} \in H} (\alpha - L(\vec{x}, \vec{\beta})) \prod_{\vec{x} \in H} (\alpha - T(\vec{x}, \vec{\beta}))$$

11.2.1 Multi-column and aggregation:

Let R be the contribution of the column aggregation. In the default multivariate setting of this section, we take $R = \log(S)$. If the implementation explicitly overrides the aggregation to be univariate, then we instead take $R = S$. Thus:

$$R = \begin{cases} S & \text{if the aggregation is univariate,} \\ \log(S) & \text{if the aggregation is multivariate.} \end{cases}$$

Then the lookup soundness error is bounded by

$$\epsilon_{\mathsf{sum}} \leq \frac{KHR}{F}.$$

For the single-column case, we take $R = 1$.

To prove consistency with $\mathbf{L}$ and $\mathbf{T}$, we also account for the GKR soundness term. Writing ϵ_{gkr} for that contribution, we use

$$\epsilon_{\mathsf{gkr}} \leq \frac{1}{2} \frac{(\log(H) + \log(K))(3(\log(H) + \log(K)) + 1)}{F},$$

which matches the implementation with $H = H_L + H_T$.

11.3 Grinding

For both cases we capture the option to decrease the soundness error of the lookup round through grinding. We consider a parameter $\mathsf{grinding_bits_lookup}$ denoting the rounds of proof of work, and compute the soundness error as

$$\epsilon = \begin{cases} \epsilon_{\mathsf{sum}} 2^{-\mathsf{grinding_bits_lookup}} & \text{in the univariate case,} \\ (\epsilon_{\mathsf{sum}} + \epsilon_{\mathsf{gkr}} + \epsilon_{\mathsf{reduction}}) 2^{-\mathsf{grinding_bits_lookup}} & \text{in the multivariate case.} \end{cases}$$

References

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. Stir: Reed–solomon proximity testing with fewer queries. Cryptology ePrint Archive, Paper 2024/390, 2024. URL: https://eprint.iacr.org/2024/390.

[Gru24] Angus Gruen. Some improvements for the PIOP for ZeroCheck. Cryptology ePrint Archive, Paper 2024/108, 2024. URL: https://eprint.iacr.org/2024/108.

[Hab22a] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive*, 2022.

[Hab22b] Ulrich Haböck. A summary on the fri low degree test. Cryptology ePrint Archive, Paper 2022/1216, 2022. URL: https://eprint.iacr.org/archive/2022/1216/20241217:162441, arXiv:2022/1216.